Ders 8 Çalışma Özeti

Ders 8 Çalışma Özeti

Genel Konular

- Dersin biçimi: öğrenci sunumları
  - Chris Richardson'ın "Microservices Patterns" kitabından iki bölüm sunulur: Bölüm 3 "Interprocess Communication in a Microservice Architecture" (mikroservis mimarisinde süreçler arası iletişim) ve Bölüm 4 "Managing Transactions with Sagas" (saga ile transaction yönetimi). İlk sunum önceden kaydedilmiş bir video olarak izletilir. Hoca her sunumun ardından özet ve değerlendirme yapar.
- Bölüm 3: Etkileşim stilleri (ekrandan, 00:04:50)
  - Birinci boyut: bire bir (one-to-one; her istemci isteğini tam bir servis işler) ya da bire çok (one-to-many; her isteği birden çok servis işler).
  - İkinci boyut: senkron (istemci servisten zamanında yanıt bekler, beklerken bloke olabilir) ya da asenkron (istemci bloke olmaz, yanıt varsa hemen gelmek zorunda değildir).
  - Slayttaki not: senkron istek/yanıt etkileşim stili, kullanılan IPC teknolojisinden büyük ölçüde bağımsızdır (orthogonal).
- Bölüm 3: API tanımı ve evrimi (ekrandan, 00:07:25)
  - API tasarımına bir arayüz tanımıyla (interface definition language, IDL) başlamak gerekir. Mikroservis uygulamalarında API'yi değiştirmek zordur, çünkü bir servisin API'si birçok istemci tarafından kullanılır.
  - Anlamsal sürümleme (semantic versioning, SemVer): sürüm numarası MAJOR.MINOR.PATCH biçimindedir. MAJOR: API'ye uyumsuz değişiklik. MINOR: geriye uyumlu iyileştirme. PATCH: geriye uyumlu hata düzeltmesi.
- Bölüm 3: Mesaj biçimleri (ekrandan, 00:10:00 ve 00:11:00)
  - Metin tabanlı: JSON ve XML. Kendini tanımlar, insan tarafından okunabilir, şemadaki birçok değişiklik geriye uyumlu yapılabilir. Sakıncası: mesajlar uzun olur (özellikle XML) ve metni ayrıştırma (parsing) maliyeti vardır.
  - İkili (binary): Protocol Buffers ve Avro. İkisi de mesaj yapısını tanımlamak için tipli bir IDL sunar. Fark: Protocol Buffers etiketli alanlar (tagged fields) kullanır, Avro tüketicisi mesajı yorumlamak için şemayı bilmek zorundadır. Verim ve performans önemliyse ikili biçim düşünülür.
- Bölüm 3: Senkron uzak yordam çağrısı (RPI) kalıbı (ekrandan, 00:11:55)
  - İstemcinin iş mantığı bir proxy arayüzünü çağırır; RPI proxy isteği servis tarafındaki RPI sunucusuna yollar; sunucu servis arayüzü üzerinden iş mantığını çağırır ve yanıtı döndürür. İstemci ve servis altıgen (hexagonal) mimariyle çizilir.
- Bölüm 3: REST (ekrandan, 00:12:55 ve 00:14:45)
  - Leonard Richardson'ın REST olgunluk modeli: Seviye 0 (servisin tek URL uç noktasına HTTP POST), Seviye 1 (kaynak üzerinde işlem için işlemi ve parametreleri belirten POST), Seviye 2 (GET istekleri için önbellekleme gibi web altyapısını kullanma), Seviye 3 (GET ile dönen kaynağın gösterimi, o kaynak üzerinde işlem yapmak için bağlantılar içerir).
  - REST kendi içinde bir IDL barındırmaz; bu yönüyle CORBA ve SOAP'tan ayrılır.
  - Yararları: basit ve tanıdık; HTTP API tarayıcıdan ya da Postman'den test edilebilir; istek/yanıt iletişimini doğrudan destekler.
  - Sakıncaları: yalnız istek/yanıt stilini destekler (sayfa yenilenmeden değişiklik gelmez, abone olma mekanizmasına uygun değildir); tek istekte birden çok kaynak getirmek zordur; birden çok güncelleme işlemini HTTP fiillerine eşlemek bazen zordur.

- Bölüm 3: gRPC (ekrandan, 00:16:30)
  - İkili mesaj tabanlı bir protokoldür; API-first (önce API) tasarıma zorlar. Mesaj biçimi Protocol Buffers'tır: verimli, kompakt; API geriye uyumlu kalarak evrilebilir.
  - Yararları: zengin güncelleme işlemleri olan API tasarlamak kolaydır; özellikle büyük mesajlarda verimli bir IPC mekanizmasıdır; çift yönlü akış (bidirectional streaming) hem RPI hem mesajlaşma stilini mümkün kılar.
- Bölüm 3: Asenkron mesajlaşma kalıbı (ekrandan, 00:17:50 ve 00:20:00)
  - Mesaj başlık (header) ve gövdeden (body) oluşur. Başlık ad-değer çiftleridir; gönderenin verdiği çiftlere ek olarak benzersiz bir mesaj kimliği (message id) ve isteğe bağlı bir dönüş adresi (yanıtın yazılacağı kanal) içerir. Gövde metin ya da ikili biçimde gönderilen veridir.
  - Mesaj türleri: belge (document; yalnız veri), komut (command; bir RPC isteğinin eşdeğeri), olay (event; gönderende kayda değer bir şey olduğunu bildirir).
  - Mesajlaşma tabanlı servis API'si belirtiminde mesaj kanallarının adları, her kanalda değişen mesaj türleri ve biçimleri (JSON, XML, Protobuf gibi bir standartla) tanımlanır. Bir servisin asenkron API'si istemcilerin çağırdığı işlemlerden ve servisin yayımladığı olaylardan oluşur.
  - Mesajlaşma için mesaj aracıları (message broker) ve bunların üzerinde çalışan kütüphane ve çerçeveler kullanılır.
- Bölüm 3: Erişilebilirliği artırmak için asenkron mesajlaşma (ekrandan, 00:23:35)
  - REST çok popüler bir IPC mekanizmasıdır ama senkron bir protokoldür: HTTP istemcisi servisin yanıtını beklemek zorundadır. Servisler senkron protokolle haberleştikçe uygulamanın erişilebilirliği (availability) düşer. Sorun REST'e özgü değildir: bir servis istemcisine ancak başka bir servisten yanıt aldıktan sonra yanıt verebiliyorsa erişilebilirlik düşer.
- Bölüm 4: Mikroservis mimarisinde transaction yönetimi (ekrandan, 00:35:10)
  - Transaction: veritabanında değişikliğe yol açan işlem. Mikroservislerde bir işlem birden çok servisin verisini günceller; servisler arası çalışan bir transaction yönetim mekanizması gerekir.
  - ACID: Atomicity (bölünmezlik), Consistency (tutarlılık), Isolation (yalıtım), Durability (kalıcılık). Slaytta Isolation'ın yanında soru işareti vardır: saga ile korunamayan özellik budur.
  - Monolitte tek veritabanı olduğu için çerçevelerin transaction API'leri yeterlidir; mikroserviste her servisin kendi veritabanı vardır.
- Bölüm 4: Dağıtık transaction ve iki aşamalı commit (ekrandan, 00:36:20)
  - 2PC: bir koordinatör, hazırlık (prepare) ve commit aşamaları. Koordinatör bütün katılımcılardan "hazır" yanıtı alırsa commit eder, biri olumsuzsa hepsini iptal eder.
  - Sorunları: MongoDB gibi modern teknolojiler dağıtık transaction'ı desteklemez; transaction'ın tamamlanması için bütün katılımcı servislerin ayakta olması gerekir, bu da erişilebilirliği düşürür.
- Bölüm 4: Saga kalıbı (ekrandan, 00:37:50 ve 00:38:50)
  - Saga: asenkron mesajlaşmayla koordine edilen bir yerel transaction dizisiyle servisler arasında veri tutarlılığını korur. Bir yerel transaction'ın tamamlanması bir sonrakini tetikler.
  - Örnek (restoran siparişi): Order Service "Create order" (Txn 1), Consumer Service "Verify consumer" (Txn 2), Kitchen Service "Create ticket" (Txn 3), Accounting Service "Authorize card" (Txn 4), Kitchen Service "Approve ticket" (Txn 5), Order Service "Approve order" (Txn 6).
  - Geri alma (rollback) yapılamaz, çünkü her yerel transaction kendi verisini commit etmiştir. Bunun yerine telafi edici transaction'lar (compensating transactions) kullanılır: T1…Tn commit edilmiş ve T(n+1) başarısızsa, Cn…C1 sondan başa doğru çalıştırılıp T1…Tn'nin etkisi geri alınır.
- Bölüm 4: Saga koordinasyonu (ekrandan, 00:40:00, 00:41:15, 00:42:50)
  - Koreografi: merkezî koordinatör yoktur; katılımcılar birbirinin olaylarına yanıt verir, transaction tamamlanana kadar olay alışverişi sürer. Yararları: basitlik, gevşek bağlılık. Sakıncaları: anlaması zordur (saga mantığı olaylara dağılmıştır), servisler arasında döngüsel bağımlılık, sıkı bağlılık riski.
  - Orkestrasyon: katılımcılara ne yapacağını söyleyen bir orkestratör sınıfı tanımlanır; iletişim komut/asenkron yanıt (command/async reply) stilindedir. Örnekte Create Order Saga orkestratörü Order Service içindedir. Yararları: daha basit (tek yönlü) ba-

ğımlılıklar, daha az bağlılık, sorumlulukların ayrılması ve basitleşen iş mantığı. Sakıncası: iş mantığının fazlasını orkestratörde toplama riski.

- Bölüm 4: Yalıtım eksikliği (ekrandan, 00:46:55, 00:48:20, 00:50:00)
  - Bir sagadaki yerel transaction commit edilince değişikliği diğer sagalar hemen görür. Bu yüzden saga ACID değil ACD modelidir (yalıtım yoktur).
  - Anomaliler: kayıp güncelleme (lost update; bir saga diğerinin yaptığı güncellemenin üzerine yazar), kirli okuma (dirty read; bir saga başka bir saganın güncellemekte olduğu veriyi okur) ve bulanık/tekrarlanamaz okuma (fuzzy/nonrepeatable read).
  - Saga yapısı: telafi edilebilir transaction'lar (compensatable), pivot transaction (sagadaki "devam/dur" noktası) ve yeniden denenebilir transaction'lar (retriable; pivottan sonra gelir, başarısı garantidir).
  - Karşı önlemler (countermeasures): semantik kilit, değişmeli (commutative) güncellemeler, karamsar görüş (pessimistic view), değeri yeniden okuma (reread value), sürüm dosyası (version file), değere göre (by value).

Hocanın Özellikle Vurguladığı Kısımlar

- RPC sıkı bağlılık yaratır, telefon hattı gibidir (00:25:56)
  - RPC (uzak yordam çağrısı) en eski ve en temel tekniktir. İki servis arasında bir iletişim kanalı açılır ve iletişim bitene kadar açık kalır; iki taraf da meşguldür ve birbirini bekler. Bu yapı gevşek değil sıkı bağlıdır (tightly coupled). Eski RMI (Remote Method Invocation) da aynı amaçla tasarlanmış bir kütüphaneydi. Servisler kilitlenir, yeni istek alınamaz.
- Mesajlaşmanın da orkestrasyonu gerekir: mesaj aracısı (00:29:16)
  - Servisler bloklamadan asenkron haberleşse bile çok örnekli (multi-instance) çalışırlar. Hangi mesajı hangi servisin alacağını, kimin yanıt beklediğini bir kuyruk mekanizması yönetmelidir: message broker ya da message queue. RabbitMQ gibi hazır çözümler vardır. Hoca bunu trafiği yöneten trafik polisine benzetir.
- Tek doğru yok, ama mümkün olduğunca asenkron (00:30:51)
  - Mesajlaşma ve servis iletişimi uygulamanın doğasına göre seçilir; bazı uygulamalarda, hatta aynı uygulamanın bazı parçalarında senkron gerekirken bazılarında asenkron gerekir. Çoğunlukla asenkron yapmaya çalışmak birbirini beklemeyen, kilitlenmeyen ve genişlemeye yatkın sistemler kurmayı kolaylaştırır; ama bazen senkronda kalmak zorunludur.
- Koreografi ve orkestrasyonun bedeli (00:53:37)
  - Koreografide herkes kendi sorumluluğunu üstlenir ama okumak, anlamak ve gerçeklemek zordur. Orkestrasyonda sorumluluk bir "orkestra şefine" verilir; bağlılık açısından daha iyidir ama sorumluluğun hangi servise verileceği sorunu çıkar ve orkestratör ayrı bir hata noktası olur: şef arıza yaparsa ya da çok istek gelip tıkanırsa ne olacak?
- Saga yalıtımı tam sağlamaz; kritik işte monolit gerekebilir (00:55:26)
  - Hoca Amazon örneği verir: stokta iki ürün varken sagalar birbirinin ara durumunu okuduğu için dört sipariş alınır, sonra ikisi iptal edilir ve müşterinin gönlü kuponla alınır. Uzaya roket yolluyorsanız ya da iş sonunda hesaba ve muhasebeye dokunuyorsa, bir yerde monolitik çalışmak gerekecektir. Her derde mikroservis çözümü yoktur; uygulama uzayına hangi mimari uygunsa o kullanılır.

Kısa Tekrar Notları

- Etkileşim stili: bire bir / bire çok ve senkron / asenkron.
- SemVer: MAJOR (uyumsuz), MINOR (geriye uyumlu iyileştirme), PATCH (geriye uyumlu hata düzeltmesi).
- Metin biçimleri: JSON, XML (okunabilir, uzun, parsing maliyeti). İkili: Protocol Buffers (etiketli alan), Avro (şema gerekli).
- RPI: proxy arayüzü - RPI proxy - RPI sunucusu - servis arayüzü.
- REST olgunluk modeli: Seviye 0 tek uç noktaya POST; Seviye 1 işlemi belirten POST; Seviye 2 GET önbellekleme; Seviye 3 yanıtta işlem bağlantıları.
- REST: basit, Postman ile test; ama yalnız istek/yanıt, çoklu kaynak ve çoklu güncelleme zor.

- gRPC: ikili, Protocol Buffers, API-first, çift yönlü akış.
- Mesaj = başlık (mesaj kimliği, dönüş adresi) + gövde. Türler: belge, komut, olay.
- Senkron iletişim erişilebilirliği düşürür.
- 2PC: koordinatör + prepare + commit; MongoDB gibi teknolojiler desteklemez, erişilebilirliği düşürür.
- Saga: asenkron mesajlaşmayla koordine edilen yerel transaction dizisi. Hata olursa telafi edici transaction'lar ters sırayla çalışır.
- Koreografi: olaylar, merkez yok. Orkestrasyon: komut/yanıt, merkezde orkestratör.
- Saga = ACD; yalıtım yok. Anomaliler: lost update, dirty read, fuzzy/nonrepeatable read.
- Compensatable - pivot - retriable.
- Karşı önlemler: semantic lock, commutative updates, pessimistic view, reread value, version file, by value.

Detaylı Açıklamalar

Süreçler arası iletişim neden önemli?

Mikroservisler kendi başına çalışan birimlerdir; birlikte iş yapabilmeleri için haberleşmeleri gerekir. Hoca bunu "servisleri konuşturmak" diye özetler. En eski teknik RPC'dir ve iki tarafı birbirine bağlayan, telefon hattı gibi açık kalan bir kanal kurar. İletişim teknik ayrıntılarla birlikte RPC kütüphanesine bırakılır, servis kendi iş mantığında (altıgen mimarinin merkezinde) kalır ve yalnız bir arayüze bağlanır (ekrandan, 00:11:55). Sorun beklemedir: iki taraf da meşgul kaldığı için servis yeni istek alamaz.

Asenkron mesajlaşmada istemci isteğini yollar ve beklemeden işine devam eder; yanıt geldiğinde onu işler. Kitabın slaytına göre servisler senkron haberleştikçe erişilebilirlik düşer: bir servis istemcisine ancak başka bir servisten yanıt aldıktan sonra yanıt verebiliyorsa, zincirdeki her servis ayakta olmak zorundadır (ekrandan, 00:23:35).

> **Ders sonrası not:** Sunumun sonunda RESTful API'nin "daha asenkron temelli" olduğu söylenir (00:24:46); hoca da REST'i "isteyelim, sonra çalışsın, ben beklemeyeyim" diye asenkron yaklaşım gibi anlatır (00:27:45). Sunumdaki slayt ise REST'in senkron bir protokol olduğunu, HTTP istemcisinin yanıtı beklemek zorunda olduğunu söyler (ekrandan, 00:23:35). Kitapta asenkron iletişim için önerilen yol büyük olasılıkla REST değil, mesaj aracısı üzerinden asenkron mesajlaşmadır.

Mesaj biçimi seçimi

Metin tabanlı biçimler (JSON, XML) kendini tanımlar ve okunabilir; XML'deki etiketler insanın mesajı kolay anlamasını sağlar. Ama mesajlar uzun olur ve her aktarımda ayrıştırma gerekir; bu, büyük mesajlarda performans sorunu doğurur (ekrandan, 00:10:00). İkili biçimler (Protocol Buffers, Avro) daha kompakttır. Biçim, mikroservisin ihtiyacına göre seçilir (ekrandan, 00:11:00).

REST ve gRPC

REST olgunluk modeli REST kullanımını seviyelere ayırır (ekrandan, 00:12:55). Seviye 0'da istemci servisin tek URL uç noktasına HTTP POST yapar. Seviye 1'de bir kaynak üzerinde işlem yapmak için işlemi ve parametreleri belirten POST istekleri vardır. Seviye 2'de GET istekleri için önbellekleme gibi web altyapısı kullanılır. Seviye 3'te bir GET isteğiyle dönen kaynak gösterimi, o kaynak üzerinde işlem yapmak için bağlantılar içerir. REST'in temel sakıncası yalnız istek/yanıt stilini desteklemesidir: sunucu tarafında bir değişiklik olunca sayfa yenilenmeden ya da arka planda istek yapılmadan değişiklik istemciye gelmez (ekrandan, 00:14:45). Hoca, bu güncelleme sorununun olay yayımlama (event publish) mekanizmalarıyla, yani ilgililerin dinlediği bir mesaj yayımlanarak çözülebileceğini söyler (00:28:57).

gRPC ikili mesaj tabanlı bir protokoldür ve Protocol Buffers kullanır. API önce tanımlanır (API-first). Zengin güncelleme işlemleri olan API tasarlamak kolaydır, büyük mesajlarda verimlidir ve çift yönlü akış sayesinde hem RPI hem mesajlaşma stilini destekler (ekrandan, 00:16:30).

Asenkron mesaj ve mesaj aracısı

Bir mesaj başlık ve gövdeden oluşur (ekrandan, 00:17:50). Sunumda başlığın önemi resim örneğiyle anlatılır: PNG ve JPEG ikisi de resimdir ama başlıkları çok farklıdır ve alıcının veriyi nasıl işleyeceği başlığa bağlıdır. Mesajlar belge, komut ya da olay olabilir. Mesajları servisler arasında taşıyan, kuyruğa dizen ve doğru servise yönlendiren yapı mesaj aracısıdır. Mesaj aracıları düşük seviyelidir ve çok kod gerektirir; bu yüzden üzerlerinde kütüphaneler ve çerçeveler kullanılır.

Saga ile transaction yönetimi

Sunumdaki e-ticaret örneğinde ödeme alınır, sonra ürün stoktan düşülür. Ödemeden sonra hata olursa işlem geri mi alınacak, devam mı edilecek? Monolitte tek veritabanı ve çerçevenin transaction API'si bunu çözer. Mikroserviste her servisin kendi veritabanı olduğu için dağıtık transaction gerekir. 2PC'de koordinatör önce bütün katılımcılara hazır olup olmadıklarını sorar, hepsi olumluysa commit aşamasına geçer ve işlemler kalıcı olur; biri olumsuzsa hepsi iptal edilir (ekrandan, 00:36:20). Ancak bu, bütün servislerin aynı anda ayakta olmasını gerektirir.

Saga, her biri tek bir serviste yerel olarak çalışıp commit eden transaction'lardan oluşur; biri bitince sonrakini tetikler. Bir adım başarısız olursa önceki adımların etkisi telafi edici transaction'larla sondan başa doğru geri alınır (ekrandan, 00:38:50). Sipariş sagasında telafi edici transaction'lar şunlardır (ekrandan, 00:48:20):

Adım	Servis	Transaction	Telafi
1	Order Service	createOrder()	rejectOrder()
2	Consumer Service	verifyConsumerDetails()	-
3	Kitchen Service	createTicket()	rejectTicket()
4	Accounting Service	authorizeCreditCard()	-
5	Restaurant Order Service	approveRestaurantOrder()	-
6	Order Service	approveOrder()	-

İlk üç adım telafi edilebilir, 4. adım (kredi kartı onayı) pivottur, son iki adım yeniden denenebilir. Kredi kartı onaylanırsa saga sonuna kadar devam eder; onaylanmazsa geri dönülüp tekrar denenmez, işlem iptal edilir.

Yalıtım eksikliğine karşı önlemler

Slayttaki karşı önlemler (ekrandan, 00:50:00):

- **Semantic lock:** uygulama seviyesinde kilit. Telafi edilebilir transaction'ın oluşturduğu ya da değiştirdiği kayda bir işaret (flag) konur; işaret kaydın henüz commit edilmediğini ve değişebileceğini gösterir. Diğer transaction'lar için kilit ya da uyarı görevi görür.
- **Commutative updates:** güncelleme işlemleri herhangi bir sırada çalıştırılabilecek şekilde tasarlanır; kayıp güncellemeyi önler.
- **Pessimistic view:** iş riskini en aza indirmek için saga adımlarının sırası değiştirilir; kirli okumaya karşıdır.
- **Reread value:** veriyi üzerine yazmadan önce yeniden okuyup değişmediği doğrulanır.
- **Version file:** bir kayda yapılan güncellemeler kaydedilir, böylece yeniden sıralanabilir; değişmeli olmayan işlemler değişmeli hâle getirilir.
- **By value:** her isteğin iş riskine göre eş zamanlılık mekanizması dinamik olarak seçilir (saga mı, dağıtık transaction mı).

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.